IT314 Software Engineering

Lab : 10

Lab Assignment : Static Code Analysis with SonarQube

Day 3

Objective :

To understand static code analysis and code quality inspection using SonarQube and the SonarQube for IDE (formerly SonarLint) extension in VS Code.

Files Provided :

Download `Lab10_Day3_Code.zip` and extract it. It contains:

File	Used in
`task1_bank_transactions.py`	Task 1
`task2_path_sum.py`	Task 2

Setup :

- Install VS Code and the **Python** extension (Python 3.8 or newer).
- From the Extensions view, install **SonarQube for IDE** by SonarSource (it was previously called SonarLint).
- Open the extracted folder in VS Code (File > Open Folder) and open each .py file. Issues appear underlined in the editor and are listed in the **Problems** panel. Type `sonarqube` in the Problems panel's filter box to hide messages from other extensions (such as Pylance). Click an issue to see its rule key and description.
- No SonarQube server, account or connected mode is needed. Use the extension as installed, with its default rules.

Task 1 :

Buggy Python Code: `task1_bank_transactions.py`

The file contains a set of utility functions for a small **banking and transactions system** (deposits, withdrawals, transfers, interest, statements and so on). Where a function has a docstring, the docstring describes what the function is **supposed** to do.

Run SonarQube for IDE on the file and **correct all warnings and logic errors**, i.e.:

- **every issue SonarQube for IDE reports** for the file (Bugs, Vulnerabilities and Code Smells), and
- **every logic error**: a function that does not do what its docstring says. The tool does not report these; find them by reading and running the code.

Rules :

- Fix the problem, do not hide it. Deleting a function, commenting out code or adding suppression comments (e.g. `# NOSONAR`) only to silence a warning is not accepted.
- Every function must keep its intended purpose. For a security issue (for example a weak hash, an insecure random generator or a disabled certificate check), switch to the secure alternative instead of removing the feature.

- Code that can never run, or values that are never used, should be removed.
- If you believe a warning should stay, leave it and explain why in your report.

Submission (Task 1) :

Provide the corrected version of the code along with a documentation report that clearly mentions the identified errors and their corresponding fixes based on your analysis (format given below).

Task 2 :

Problem Description :

- You are given the root of a binary tree. Your task is to find the **maximum root-to-leaf path sum** of the tree and determine whether there is a **root-to-leaf path whose values add up to a given target**.
- A **root-to-leaf path** starts at the root and ends at a **leaf** (a node with no children). Node values may be **negative**.
- The **maximum root-to-leaf path sum** is the largest sum of node values over all root-to-leaf paths. For an empty tree, return `None`.
- **Path sum check:** return True if at least one root-to-leaf path has a sum exactly equal to the target, otherwise False. A path that reaches the target before it reaches a leaf does not count. An empty tree returns False.

> Example: for [5, 4, 8, 11, null, 13, 4, 7, 2] the maximum root-to-leaf sum is 27 (5 + 4 + 11 + 7) and a path with sum 22 exists (5 + 4 + 11 + 2).
>
> Trees are written in level order; `null` means a missing child.

Code :

`task2_path_sum.py` (in the zip)

Task Description :

- Run SonarQube for IDE analysis on this code.
- Identify and fix all issues reported by SonarQube for IDE and all logic errors (wrong results, or crashes when the code runs).
- Keep the function names `max_path_sum(node)` and `has_path_sum(root, target)` unchanged. After fixing, `python task2_path_sum.py` must run without errors.
- Design test cases that check the corrected functions, including the test cases where the original code fails.

Submission (Task 2) :

Provide the corrected version of the code along with a documentation report that clearly mentions the identified errors, their corresponding fixes, and the test cases where the original code fails, based on your analysis.

Submission Format :

Submit **one zip file** named `<StudentID>_Lab10.zip` (for example `202301234_Lab10.zip`) with exactly this structure:

```
202301234_Lab10/
|-- task1_bank_transactions.py    corrected Task 1 code
|-- task2_path_sum.py             corrected Task 2 code
`-- 202301234_Lab10_Report.pdf    documentation report
```

- Keep the original file names for the corrected code files.
- The report must be a **PDF** named `<StudentID>_Lab10_Report.pdf`.
- Upload only the one zip file. Do not upload the code and report separately.

Report Structure :

1. **Cover:** name, student ID and lab day (Day 3).
2. **Task 1:** (a) screenshot of the Problems panel (filtered to SonarQube) **before** fixing; (b) issue table; (c) screenshot **after** fixing, showing no SonarQube issues left; (d) any issue you kept, with the reason.
3. **Task 2:** (a) screenshot before fixing; (b) issue table; (c) test case table; (d) screenshot after fixing.
4. Screenshots must show your own VS Code window with the file name visible. If the issue list is long, use more than one screenshot.

Issue table (Task 1 and Task 2) :

One row per issue. **Rule**, **Type** (Bug, Vulnerability or Code Smell) and **Impact** are copied from what SonarQube for IDE shows for the issue. For a logic error you found yourself, write Type = Logic Error and put '-' for Rule and Impact.

No.	Line(s)	Rule	Issue (tool message or your description)	Type	Impact	Fix applied
1	42	python:S1481	Remove the unused local variable 'x'.	Code Smell	Maintainability: Low	Removed the variable.
2	57	-	mean([2, 4, 6]) returns 3; docstring says it returns the average (4).	Logic Error	-	Divide by len(values) instead of len(values) + 1.
3	...	...	...	...	...	...

Test case table (Task 2) :

Write at least **6** test cases; at least **4** of them must be cases where the original code fails. If the original code crashes, write the error (e.g. TypeError) as its output, and in the last column name the logic bug that would still make the test fail after the crash is fixed.

Test ID	Input tree (level order)	Function	Expected	Original output	Corrected output	Bug exposed
T1	...	...	...	...	...	...